and the highest level is called `Fatal` (instead of `Critical`). You will use the `Log.Fatal` method to record details of any exceptions that prevented the application from starting.

Now that you have replaced the existing log providers with Serilog, it's time to test that the replacement works. Remove the line that throws the exception in the `OnGetAsync` method of the main `IndexModel`, and then run the application. You should see something along the lines of the following listing in the console.

Listing 12.20 The Serilog console logger output

```
[08:35:58 INF] Starting up
[08:35:59 INF] Now listening on: https://localhost:7236
[08:35:59 INF] Now listening on: http://localhost:5236
[08:35:59 INF] Application started. Press Ctrl+C to shut down.
[08:35:59 INF] Hosting environment: Development
[08:35:59 INF] Content root path: D:\repos\
➥ Razor-Pages-In-Action\Chapter12\Serilog\CityBreaks\
[08:36:00 INF] 12 Cities retrieved
```

Notice the difference from the default loggers. There is no category included in the log message. You can change that by passing a string to the `outputTemplate` parameter of the `WriteTo.Console` method (http://mng.bz/M0WB). Serilog refers to the category as the `SourceContext`. The following template includes the category and uses the `NewLine` property to place the log message over multiple lines:

```
"[{Timestamp:HH:mm:ss} {Level:u3}]
➥ {SourceContext}{NewLine}{Message:lj}{NewLine}{Exception}"
```

Of more interest from a structured log point of view is the contents of the log file created in the Logs folder. This should now contain a number of logs in JSON format, each one on a separate line. The file content as a whole is not